

Aprendizaje por Refuerzo Métodos Computacionales

Daniela Pucci de Farias

Universidad Torcuato Di Tella

June 2025

Clase 4

- ▶ Computación en MDPs
 - ▶ Iteración de valor
 - ▶ Iteración de política
- ▶ Desafíos en MDPs
 - ▶ Incertidumbre y/o complejidad del ambiente
 - ▶ “Curse of dimensionality” (la maldición de la dimensionalidad)
- ▶ Métodos de Aprendizaje por Refuerzo
 - ▶ Simulación Monte Carlo
 - ▶ TD-Learning
 - ▶ Q-Learning

Revisión: MDPs

- ▶ MDP: tupla (S, A, P, r)
- ▶ Ecuación de Bellman y función valor:

$$V^*(x) = \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V^*(y) \right\}$$

- ▶ Función de políticas óptimas:

$$u^*(x) = \arg \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V^*(y) \right\}$$

- ▶ Algoritmos para resolver la ecuación de Bellman:
 - ▶ iteración de valor
 - ▶ iteración de política

Iteración de Valor

1. Hacemos $t = 0$. Inicializamos $V_0(x)$ con un valor arbitrario, por ejemplo $V_0(x) = 0 \forall x$.
2. Hacemos $t = t + 1$. Calculamos
$$V_{t+1}(x) = \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V_t(y) \right\}.$$
3. Utilizando un criterio de convergencia para V_t , decidimos si parar o volver al paso 2.
 - Por ejemplo, $\max_x |V_t(x) - V_{t-1}(x)| < \epsilon$ para algún $\epsilon > 0$

Iteración de Política

1. Hacemos $k = 0$. Empezamos con una política arbitraria u_0 .
2. Evaluamos la función valor V_{u_k} asociada con la política u_k . Para eso, debemos resolver:

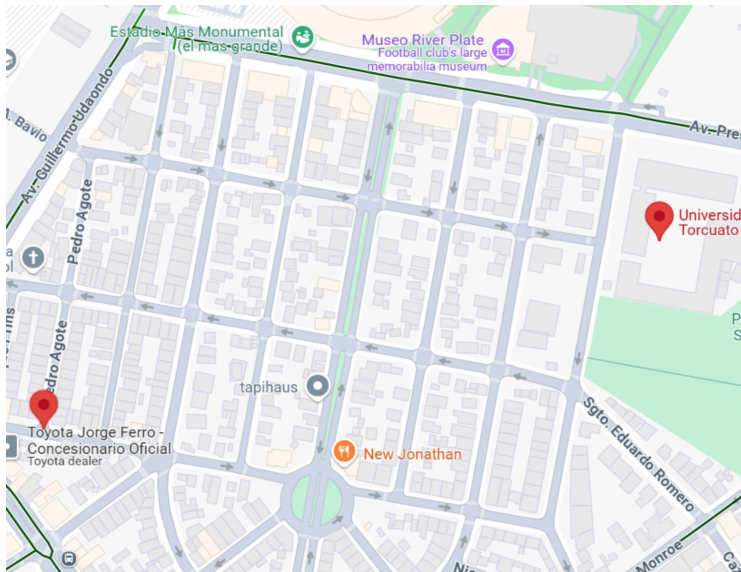
$$V_{u_k}(x) = r_{u_k}(x) + \alpha \sum_y P_{u_k}(x, y) V_{u_k}(y)$$

Calculamos una nueva política u_{k+1} , que será la **política codiciosa** con respecto a V_{u_k} :

$$u_{k+1}(x) = \arg \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V_{u_k}(y) \right\}$$

3. Si $u_k = u_{k+1}$, paramos. Si no, hacemos $t = t + 1$ y volvemos al paso 2.

Ejemplo: Ruta de Casa a La Universidad



Ejemplo: Ruta de Casa a La Universidad

- ▶ Modelo MDP:

- ▶ Estado = posición en el mapa, en las esquinas $x = (i, j)$
- ▶ Acciones = desplazamiento, una cuadra por vez:

$$A = \{(0, 1), (0, -1), (-1, 0), (1, 0)\} \equiv \{N, S, O, E\}$$

- ▶ Transición (determinística):

$$x_{t+1} = \begin{cases} x + a & \text{if } x_t \neq x^* \\ x^* & \text{if } x_t = x^* \end{cases}$$

- ▶ Recompensa (castigo, en este caso):

$$r_a(x) = \begin{cases} E_w [\text{tiempo}(x, x + a, w)] & \text{if } x \neq x^* \\ 0 & \text{if } x = x^* \end{cases}$$

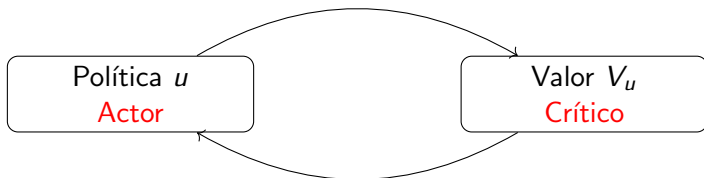
- ▶ Criterio de optimalidad:

$$\min_u E \left[\sum_{t=0}^{\infty} r_u(x_t) \right].$$

- ▶ Implementación en Python de iteración de valor e iteración de políticas

Interpretación

- ▶ Diagrama para iteración de política



- ▶ Algoritmo actor-crítico
 - ▶ **Actor** selecciona una política u_k (por ejemplo, codiciosa con respecto a $V_{u_{k-1}}$;
 - ▶ **Crítico** evalúa la política, generando la función valor V_{u_k}
- ▶ En iteración de política, V_u es la función valor exacta de u .
- ▶ Se puede reemplazar V_u con una aproximación $\tilde{V}_u$. Muchos algoritmos de RL tienen esa forma.
- ▶ Iteración de valor puede ser interpretada como un caso límite de actor-crítico:
 - ▶ u_k es la política codiciosa con respecto a V_{k-1}
 - ▶ la evaluación se vuelve
$$\tilde{V}_{u_k}(x) \equiv V_k = r_{u_k}(x) + \alpha \sum_y P_{u_k}(x, y) V_{k-1}(y)$$

Desafíos Para Aplicación de MDPs

- ▶ MDPs representan un paradigma que permite modelar y, en principio, encontrar estrategias óptimas para una amplia gama de situaciones.
- ▶ Pero... en la práctica hay varios desafíos.
- ▶ Ejemplos:
 - ▶ Precificación personalizada: en principio, un e-commerce podría ajustar precios para cada usuario individualmente
 - ▶ Probabilidad de que el usuario acepte un determinado precio no es conocida de antemano
 - ▶ Gestión de relaciones con clientes: tomar decisiones de contacto o retención según el estado del cliente
 - ▶ El estado emocional del cliente (satisfecho, dudoso, a punto de irse) no se puede observar directamente, solo hay observaciones indirectas
 - ▶ Gestión de inventario con múltiples productos:
 - ▶ Tanto el espacio de estados (nivel de inventario de cada producto) y de acciones (cuánto más pedir de cada producto) crecen exponencialmente con el número de productos

Desafíos Para la Aplicación de MDPs

Información Parcial

1. Parámetros del modelo desconocidos: probabilidades de transición y recompensas $\Rightarrow$ **aprendizaje**
2. Estado x no siempre puede ser observado.
 - ▶ observaciones $h(x, w)$ con ruido w
 - ▶ POMDP: proceso markoviano de decisión parcialmente observable $\Rightarrow$ **en principio, se convierte en MDP; en la práctica, muy difícil**

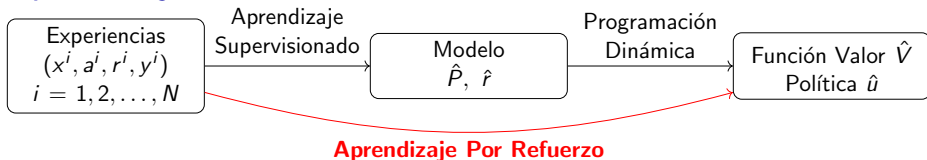
Complejidad del Sistema

- 3 “Maldición de la dimensionalidad”
 - ▶ Número de estados exponencial en el número de variables de estado
 - ▶ Dificultades para almacenar P y r y calcular V^*
 - ▶ Número de acciones exponencial en el número de variables de acción
 - ▶
$$\max_{a \in A} \left\{ r_a(x) + \sum_y P_a(x, y) V^*(y) \right\}$$ $\Rightarrow$ **generalización**

Desafío: Incertidumbre Sobre Parámetros del Modelo

- ▶ La metodología está basada en información que no siempre tenemos, en especial las probabilidades de transición P y recompensas r .
- ▶ Qué hacer?
- ▶ "Aprender" a partir de interacciones con el ambiente.
- ▶ Con qué datos?
 - ▶ trayectorias
 - ▶ solo una: $(x_0, a_0, r_0, x_1, a_1, r_1, \dots, x_N)$ (N puede ser infinito)
 - ▶ multiples: $(x_0^i, a_0^i, r_0^i, x_1^i, a_1^i, r_1^i, \dots, x_N^i)$, $i = 1, \dots, M$
 - ▶ tuplas (x^i, a^i, r^i, y^i) , $i = 1, 2, \dots, N$

Aprendizaje Por Refuerzo



- ▶ Cómo ir de los datos (x^i, a^i, r^i, y^i) , $i = \dots, N$ a una política de decisión $\hat{u}$?
- ▶ **Aprendiendo un modelo**
 1. Aprendizaje supervisado.
 - ▶ Con $I_x = \{i : x^i = x\}$ y $I_{xy} = \{i : x^i = x, y^i = y\}$, podemos estimar: $\hat{P}(x, y) = \frac{|I_{xy}|}{|I_x|}$, $\hat{r}(x) = \frac{\sum_{i \in I_x} r_i}{|I_x|}$.
 - ▶ Alternativamente, podemos estimar parámetros del sistema que permiten calcular las transiciones y recompensas (función precio x demanda, retorno y volatilidad de un activo, etc.)
 2. Programación dinámica con $\hat{P}, \hat{r} \Rightarrow \hat{V}, \hat{u}$
- ▶ **Sin modelo**
 - ▶ Aprender $\hat{V}$ y/o $\hat{u}$ directamente de los datos
 $\Rightarrow$ **aprendizaje por refuerzo**

Aprendizaje por Refuerzo: ¿Por Qué?

- ▶ **Esencial cuando no es posible construir un modelo explícito del ambiente**
 - ▶ Hay que aprender sobre el ambiente **mientras se actúa** en él
 - ▶ Ejemplo: interacción secuencial con usuarios, robots, mercados
 - ▶ Situación enfrentada también por humanos y animales
 - ▶ El comportamiento del ambiente puede ser **cambiante**
 - ▶ No conviene asumir una dinámica fija
- ▶ **Reduce suposiciones fuertes sobre la dinámica → mayor robustez**
 - ▶ Aprende directamente de la experiencia, sin modelo paramétrico
- ▶ **También útil cuando se dispone de un modelo, pero...**
 - ▶ El modelo es demasiado complejo para resolver analíticamente (por ejemplo, no lineal o estocástico)
 - ▶ El sistema tiene dimensión muy alta → “maldición de la dimensionalidad”
 - ▶ El modelo solo está disponible como un **simulador** (caja negra)
 - ▶ Ejemplo: videojuegos, simuladores de procesos industriales o financieros

Aprendizaje Por Refuerzo: Cómo?



1. Optimización directa de la política

- ▶ Consideramos $\hat{u}(x) = u(x, \psi)$ donde $\psi \in \mathbb{R}^K$ son parámetros a calibrar
 - ▶ Comprar más mercadería en cantidad $u(x, \psi) = \psi_0 \mathbf{1}(x \leq \psi_1)$
 - ▶ Ofrecer pasaje de avión a precio $u(x, \psi) = \psi_0 + \psi_1 t + \psi_2 x$
 - ▶ Bid para publicidad online: estado $x = (p, v) =$ estimativa actual de (probabilidad p de clic, valor esperado v de compras resultantes del clic), bid $= u(x, \psi) = \psi_0 + \psi_1 p + \psi_2 v$
 - ▶ Estos ejemplos utilizan parametrizaciones lineales, pero podrían también ser no lineales, incluyendo redes neuronales.
- ▶ Resolvemos $\max_{\psi \in \mathbb{R}^K} \mathbb{E} \left[\sum_{t=0}^{\infty} \alpha^t r_{u(x_t, \psi)}(x_t) \right] = \max_{\psi \in \mathbb{R}^K} f(\psi)$.
- ▶ Utilizamos descenso estocástico de gradientes para realizar la optimización.

Aprendizaje Por Refuerzo: Cómo?



2. Estimación de la función valor

- ▶ Aprendizaje supervisado/Monte Carlo
- ▶ Versiones de iteración de valor o iteración de políticas

3. Métodos actor-crítico

- ▶ Combinan $\hat{u}(x) = u(x, \psi)$ (actor) con estimación/aproximación de la función valor $\hat{V}$ (crítico)
- ▶ Utilizados para bajar la variancia de optimización directa de la política

Estimación de La Función Valor

- ▶ La función valor es definida implícitamente por la ecuación de Bellman:

$$V^*(x) = \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V^*(y) \right\}$$

- ▶ Cómo estimarla a partir de datos?
- ▶ Empezamos con el caso de una política fija
 - ▶ proceso markoviano con recompensa (MRP)

Estimación de La Función Valor: Simulación Monte Carlo

- ▶ Con una política fija u , tenemos:

$$V(x) \equiv V_u(x) = \mathbb{E} \left[\sum_{t=0}^{\infty} \alpha^t r_u(x_t) | x_t = x \right]$$

- ▶ A partir de una trayectoria $(x_0, r_0, x_1, r_1, x_2, r_2, \dots)$, podemos construir muestras (x_t, R_t) , $t = 0, 1, \dots$ de la función V :

$$R_t = \sum_{\tau=1}^{\infty} \alpha^{\tau-t} r_{\tau}$$

- ▶ Para cada estado x visitado en la trayectoria, definimos $T_x = \{t : x_t = x\}$ y tenemos:

$$\hat{V}(x) = \frac{\sum_{t \in T_x} R_t}{|T_x|}$$

Bootstrapping

- ▶ Monte-Carlo se basa en muestras (x_t, R_t) , $t = 0, 1, \dots$ de la función V , donde $R_t = \sum_{\tau=1}^{\infty} \alpha^{\tau-t} r_{\tau}$
 - ▶ Solo podemos calcular las muestras después de una larga trayectoria
 - ▶ Las muestras tienen variancia que normalmente crece con el tamaño de la trayectoria
- ▶ Podemos utilizar **bootstrapping** observando las muestras se relacionan con la función valor:

$$\begin{aligned} R_t &= \sum_{\tau=t}^{\infty} \alpha^{\tau-t} r_{\tau} \\ &= r_t + \alpha \sum_{\tau=t+1}^{\infty} \alpha^{\tau-(t+1)} r_{\tau} \\ &\approx r_t + \alpha [V(x_{t+1}) + \epsilon] \end{aligned}$$

- ▶ Podemos actualizar $\hat{V}(x_t)$ inmediatamente al observar la transición (x_t, a_t, r_t, x_{t+1}) :

$$\hat{V}(x_t) = (1 - \gamma_t) \hat{V}(x_t) + \gamma_t [r_t + \alpha \hat{V}(x_{t+1})]$$

donde γ_t es la **tasa de aprendizaje**.

TD-Learning

Algoritmo TD-Learning

1. Inicializamos $\hat{V}$ según alguna regla y hacemos $t = 1$.
2. Con observación (x_t, r_t, x_{t+1}) , actualizar

$$\hat{V}(x_t) = \hat{V}(x_t) + \gamma_t \left[r_t + \alpha \hat{V}(x_{t+1}) - \hat{V}(x_t) \right]$$

3. Hacemos $t = t + 1$ y retornamos al paso 2.

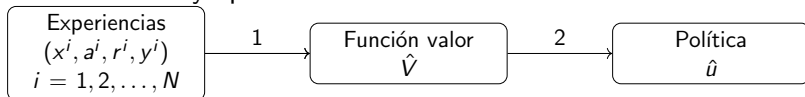
- Observamos que

$$\mathbb{E} \left[r_t + \alpha \hat{V}^{t-1}(x_{t+1}) \right] = r(x_t) + \alpha \sum_y P(x, y) \hat{V}(y)$$

- TD-learning es una versión estocástica de iteración por valor
- Las muestras (x_t, r_t, x_{t+1}) , $t = 0, 1, \dots$ no necesitan ser parte de una trayectoria!

Un Obstáculo en RL

- ▶ En este curso, nos vamos a enfocar en RL basado en estimativas y aproximaciones de la función valor.



- ▶ Conceptualmente, dos pasos:
 1. Estimamos $V^* \approx \hat{V}$ a partir de (x^i, a^i, r^i, y^i) , $i = 1, 2, \dots$.
 2. Calculamos $\hat{u}$, la política codiciosa con respecto a $\hat{V}$:

$$\hat{u}(x) = \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) \hat{V}(y) \right\}.$$

- ▶ Problema: mismo que podamos completar el paso 1), el paso 2) requiere conocimiento de P y R .
- ▶ Solución basada en la **función** Q .

La Función Q

- ▶ Recordemos la ecuación de Bellman:

$$V^*(x) = \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V^*(y) \right\}.$$

- ▶ Vamos a definir

$$Q^*(x, a) = r_a(x) + \alpha \sum_y P_a(x, y) V^*(y).$$

- ▶ Q^* satisface a otra ecuación de Bellman:

$$Q^*(x, a) = r_a(x) + \alpha \sum_y P_a(x, y) \max_{a' \in A} Q^*(y, a').$$

- ▶ Con una aproximación $Q^* \approx \hat{Q}$, podemos obtener una política:

$$\hat{u}(x) = \max_{a \in A} \hat{Q}(x, a).$$

- ▶ Podemos encontrar una aproximación $\hat{Q}$ a partir de datos utilizando TD-Learning. Con la función Q , ese método se denomina **Q-Learning**.

Q-Learning

1. Inicializar $\hat{Q}$ según alguna regla y hacemos $t = 1$.
2. Seleccionar acción a^i
3. Con observación (x^i, a^i, r^i, y^i) , actualizar

$$\hat{Q}(x^i, a^i) = \hat{Q}(x^i, a^i) + \gamma_i \left[r^i + \alpha \max_{a \in A} \hat{Q}(y^i, a) - \hat{Q}(x^i, a^i) \right]$$

4. Hacer $t = t + 1$ y retornar al paso 2.

Q-Learning - Intuición

$$\hat{Q}(x^i, a^i) = \hat{Q}(x^i, a^i) + \gamma_i \underbrace{\left[r^i + \alpha \max_{a \in A} \hat{Q}(y^i, a) - \hat{Q}(x^i, a^i) \right]}_{\text{diferencia temporal (TD)}}$$

- ▶ La actualización es intuitiva
 - ▶ la diferencia temporal indica si la observación del valor de la acción a^i en el estado x^i es mejor o peor que nuestra estimativa $Q(x_i, a_i)$
 - ▶ la estimativa es actualizada en proporción a la diferencia temporal
- ▶ Q-learning también es una versión estocástica de iteración de valor

Q-Learning - Convergencia

- Tenemos:

$$\hat{Q}(x^i, a^i) = \hat{Q}(x^i, a^i) + \gamma_i \left[r^i + \alpha \max_{a \in A} \hat{Q}(y^i, a) - \hat{Q}(x^i, a^i) \right]$$

- Definimos $I_{x,a} = \{i : (x^i, a^i) = (x, a)\}$ como el conjunto de ocurrencias de (x, a) en nuestra muestra.
- Entonces $\hat{Q} \rightarrow Q^*$ si:

$$\sum_{i \in I_{x,a}} \gamma_i = \infty, \quad \sum_{i \in I_{x,a}} \gamma_i^2 < \infty \quad \forall (x, a).$$

- Observaciones:
 - todos los pares (x, a) tienen que ser visitados "infinitas veces"
 - no es necesario visitarlos en ningún orden específico
 - convergencia a Q^* asegurada sin necesidad de seguir una política específica

Exploración vs. Explotación

- ▶ En Q-learning, para garantizar convergencia teórica, es necesario que todos los pares (x, a) sean visitados infinitas veces.
- ▶ En la práctica, debemos equilibrar **explotación** (usar lo que ya sabemos) con **exploración** (probar cosas nuevas)
- ▶ Si elegimos siempre la acción codiciosa:
 $a^i = \arg \max_{a \in A} \hat{Q}(x^i, a)$ corremos el riesgo de ignorar acciones que podrían ser mejores
- ▶ Solución clásica: **política ϵ -codiciosa**

$$a^i = \begin{cases} \arg \max_{a \in A} \hat{Q}^{i-1}(x^i, a) & \text{con probabilidad } 1 - \epsilon \\ \text{acción aleatoria} & \text{con probabilidad } \epsilon \end{cases}$$

- ▶ En general, ϵ puede decaer con el tiempo:

$$\epsilon_t = \frac{1}{1+t}, \quad \text{o bien: } \epsilon_t = \epsilon_0 \cdot \beta^t$$

- ▶ ¿Cuándo es conveniente mantener un poco de exploración incluso al final del entrenamiento?

Tasa de Aprendizaje

$$\hat{Q}(x^i, a^i) = \hat{Q}(x^i, a^i) + \gamma_i \left[r^i + \alpha \max_{a \in A} \hat{Q}(y^i, a) - \hat{Q}(x^i, a^i) \right]$$

- ▶ **¿Qué hace la tasa de aprendizaje γ_i ?**
 - ▶ Controla cuánto pesan las nuevas observaciones frente a lo aprendido previamente
- ▶ Para garantizar convergencia debemos tener:
 $\sum_{i \in I_{x,a}} \gamma_i = \infty, \sum_{i \in I_{x,a}} \gamma_i^2 < \infty \quad \forall (x, a).$
- ▶ **Si γ_i decae demasiado lentamente:**
 - ▶ Aprendizaje rápido, pero inestable
 - ▶ Q puede oscilar mucho con recompensas ruidosas
- ▶ **Si γ_i decae demasiado rápido:**
 - ▶ Aprendizaje lento, puede estancarse
 - ▶ Difícil adaptarse si el entorno cambia
- ▶ $\alpha_t(x, a) = \frac{1}{1+N_t(x,a)}$ satisface a esas condiciones. En la práctica, otras opciones pueden ser mejores.
- ▶ ¿Cuándo sería útil mantener una tasa de aprendizaje constante en lugar de decreciente?